0238. Product of Array Except Self

Medium · Arrays & Hashing

leetcode.com/problems/product-of-array-except-self

Problem

- Return answer where answer[i] is the product of all nums except nums[i].
- Must run in O(n) and without using division.
- Example: [1,2,3,4] → [24,12,8,6].

Intuition

- Answer[i] = (product of everything left of i) (product of everything right of i).
- Prefix products from the left and suffix products from the right give both halves.
- We can store prefixes in the output array and fold the suffix in with a single running variable.

Brute Force

- For every i multiply all other elements: O(n^2).
- Division (total / nums[i]) is banned and also breaks on zeros.

Optimal Approach

- Pass 1 (left to right): res[i] = product of nums[0..i-1]; start with 1.
- Pass 2 (right to left): keep suffix = product of nums[i+1..]; res[i] = suffix; then suffix = nums[i].
- Output array does not count as extra space, so space is O(1).

Worked Example

```
nums = [1, 2, 3, 4].

pass 1 prefix : res = [1, 1, 2, 6]
pass 2 suffix : i=3 res=6*1=6   suffix=4
                i=2 res=2*4=8   suffix=12
                i=1 res=1*12=12 suffix=24
                i=0 res=1*24=24
answer = [24, 12, 8, 6]
```

Java Solution

```java
public class ProductOfArrayExceptSelf {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        res[0] = 1;
        for (int i = 1; i < n; i++) res[i] = res[i - 1] * nums[i - 1];
        int suffix = 1;
        for (int i = n - 1; i >= 0; i--) {
            res[i] *= suffix;
            suffix *= nums[i];
        }
        return res;
    }
}
```

Complexity

- Time: O(n), two passes.
- Space: O(1) extra (excluding output).

Pitfalls

- Off-by-one: prefix for index i excludes nums[i]; update the running product after using it.
- Zeros are handled naturally; no special-casing needed.